

SmartJourney: Testing Summary for the Progress Evaluation

Source: SmartJourney_Master_Test_Plan.docx (prepared 2026-09-27). Every number below was measured by running the suite, not estimated.

1. The 30-second version (say this first)

SmartJourney is three separate services: a Next.js frontend, a NestJS backend and a FastAPI/LangGraph AI backend. We therefore tested the **boundaries between services** as well as each service. We have **~630 automated tests across all three repos, all passing**, running in **GitHub Actions CI on every push**. On top of that we ran a **12-scenario live "golden" test of the AI planner (10/12 passed)**, a **load test**, a **page-performance check** and **security/ownership tests**. Where something failed or could not be run, we report it honestly instead of rounding up.

2. Why the testing is shaped this way

- The system is not one MVC app, so the mission is to test every service boundary: browser → NestJS, NestJS → FastAPI, FastAPI → LLM and external APIs, and NestJS → Postgres/PostGIS.
 - The AI output is **not deterministic**, so "correct" means *schema-valid, within budget, and policy-compliant*, with a tested fallback when the LLM is down. We never assert exact text.
 - The top security concern is **data isolation**: one traveler must never see another's itineraries, expenses or chat.
 - Out of scope: the Flutter mobile app (not started) and billing (not built).
-

3. Tools used, by testing type

Testing type	What it checks	Tools
Data & DB integrity	Prisma schema matches the raw-SQL migrations; every query is filtered by <code>user_id</code>	Vitest, hand-written Prisma fakes, Prisma Studio, psql, PostGIS (Docker)
Function / unit	Business logic: budget status, ownership, idempotency, error mapping	Vitest 4 (backend), pytest 9.1.1 (AI backend), <code>@vitest/coverage-v8</code>
UI	Route protection, Keycloak token refresh, component rendering, user journeys	React Testing Library , jsdom, Playwright
Performance profiling	Page-load cost and API latency	Chrome DevTools , Lighthouse , NestJS request logs
Load	Behaviour under concurrent users	autocannon (run via npx)
Security & access control	Ownership, role gating, 401/403/404 behaviour	Vitest, supertest , jsonwebtoken (signed test tokens), manual network-exposure check
Failover & recovery	Graceful degradation when the LLM or an API is down	pytest, <code>scripts/e2e_check.py</code> , <code>scripts/check_llm_chain_reliability.py</code>
Configuration / CI	Tests pass from a clean checkout with no secrets	GitHub Actions (3 workflows), Node 26 / Node 22, Python 3.12
Code inspection	Lint cleanliness	oxlint (backend), next lint / ESLint (frontend)
Auth infrastructure	Role setup	Keycloak admin console (realm roles <code>admin</code> and <code>traveler</code>)

4. How each type was actually done

4.1 Data & database integrity

- Each backend service spec builds a **hand-rolled Prisma fake** and asserts the exact `where / data` object passed to it.
- Example: `budget.service.spec.ts` asserts the itinerary lookup always filters by **both `id` and `user_id`**.
- Schema drift is caught with `prisma db pull` and a diff against the checked-in schema.
- Caveat: migrations are raw SQL, not Prisma Migrate, and the runner can fail silently, so its exit code is always checked.

4.2 Function (unit) testing

- **Backend: 69 tests in 14 files, all passing.**
- **AI backend: 544 tests in 42 files, all passing.** Run as `python -m pytest -m "not external" -q`.

- **No unit test makes a real network or LLM call.** The LLM is always faked, and we verified this by running the suite with no Docker containers up.
- Lesson learned: some tests silently depended on API keys from the local `.env`, so CI now supplies placeholder keys.

4.3 UI testing

- **React Testing Library: 18 tests in 5 files, all passing.** They cover middleware role gating, the Keycloak JWT-refresh callback, the trip mapper, the trip store and one component. No backend is needed.
- **Playwright (journey level): 4 scenarios. Only 1 was verified passing** (unauthenticated user is redirected). The other **3 are written but skipped** because they need a seeded Keycloak session, and we have no headless way to get one yet. Being upfront about that is deliberate.

4.4 Performance profiling

- Chrome DevTools baseline on `/home`: **31 requests, 2.8 MB, DOMContentLoaded 211 ms, full load 600 ms.**
- This was measured on the dev server, not a production build, so it is a reference point and not an SLA.
- The only enforced number is the **120 s NestJS → AI-backend timeout**. Typical `/trip-plan` latency is 5–20 s.
- This is our **least mature** area, and the plan flags it as a gap.

4.5 Load testing

- One manual run: `npx autocannon -c 20 -d 30 http://localhost:3001/listings` (20 connections, 30 s).
- Result: **avg 394 req/s, p50 50 ms, p99 68 ms, max 145 ms, ~12k requests in 30 s.**
- It covers one route, run locally, so it is a baseline and not a capacity claim.
- The real bottleneck is **external API quotas**, not our compute: Nominatim 1 req/s, OpenRouteService 500/day, and Groq's free-tier tokens per minute. For that reason we mock those APIs in load tests rather than burn quota.

4.6 Security & access control

- Every service spec has a "**foreign user gets NotFoundException**" case. It returns 404, not 403, so a foreign resource looks the same as one that doesn't exist.
- `roles.guard.spec.ts` covers the role decision table.
- **6 backend e2e tests** run the real `JwtAuthGuard → RolesGuard → CurrentUser` chain with a signed JWT. They check 401 with no token, 403 for a traveler hitting `/admin/stats`, and 404 for the wrong owner.
- **Residual risk:** the AI backend has no authentication of its own. NestJS is the only trust boundary, and the AI backend must stay network-isolated inside Docker. That isolation is checked manually, not by an automated test.

4.7 Failover & recovery

- `ai-backend.service.spec.ts` checks that a non-2xx response, a timeout and a network error from the AI backend all map to **502** and never to an unhandled exception.
- The **golden-scenario harness** (`scripts/e2e_check.py`) runs 12 realistic scenarios against the live stack. It includes "Gemini unavailable", which must return a fallback plan with HTTP 200.
- **Result: 10/12 (83.3%)**. The target is $\geq 11/12$.
- The first attempt scored 8/12 because the **test harness itself was broken** (stale mock-patch targets). We fixed it.
- Scenario 5 ("make day 2 cheaper") failed its "day-2 cost strictly lower" check.
- Scenario 6 (follow-up "I'm starting from Polonnaruwa") hit an **unhandled exception**, which is an open orchestrator defect (see §6).

4.8 Configuration / CI

- **Three GitHub Actions workflows**, one per repo, each doing a clean checkout, install and test with no Docker and no real secrets.
- **All three failed on their first push** for configuration reasons, and are now green:
- Backend moved to Node 26, because Node 22's npm 10 rejected the npm-11 lockfile.
- AI backend: run `python -m pytest`, add `respx`, `itsdangerous` and `pytest-asyncio`, and supply placeholder keys.
- Frontend: the push trigger now covers the `new-main` branch.
- Coverage is **reported, not gated**. A threshold on a suite this young would fail the build for the wrong reason.

5. Results table (as run, 2026-09-27)

Repo	Command	Result
backend	<code>npm run lint</code>	0 errors (oxlint)
backend	<code>npm test</code>	69 passed , 14 files
backend	<code>npm run test:cov</code>	70.37% statements / 53.35% branches / 62.58% functions / 72.30% lines
backend	<code>npm run test:e2e</code>	6 passed
backend	autocannon	394 req/s, p99 68 ms
frontend-web	<code>npm run lint</code>	0 errors/warnings
frontend-web	<code>npm test</code>	18 passed , 5 files
frontend-web	<code>npm run test:cov</code>	75.34% statements / 65.90% branches / 83.33% functions / 76.92% lines
frontend-web	<code>npx playwright test</code>	1 passed, 3 skipped (need a Keycloak session). From the 2026-09-26 run
ai-backend	<code>python -m pytest -m "not external" -q</code>	544 passed , 42 files
ai-backend	<code>scripts/e2e_check.py</code>	10/12 (83.3%) golden scenarios, live

Totals: 69 + 6 + 18 + 544 = **637 automated tests, 0 failing**. This round added 87 new tests: 38 backend unit, 6 backend e2e, 18 frontend unit and 4 Playwright. The 544 AI-backend tests were already there.

6. Real defects the testing found (good evidence that testing works)

- Prisma date filter bug (fixed).** `searchEvents` built its `where` clause with two object spreads that both used the key `start_datetime`. The second silently overwrote the first, so a request with both bounds lost its "from" filter. The date-window unit test caught it, and the fix merges both bounds into one spread.
- Broken golden-scenario harness (fixed).** Scenarios 7–9 patched functions that a refactor had moved out of the registry, so they crashed before testing anything. They now patch where the functions are used.
- CI failures on first push (fixed).** All three repos failed for configuration reasons, as described in §4.8.
- Orchestrator follow-up crash (open at time of writing).** On a follow-up turn where the planner fails (here after Gemini 504 timeouts), `_verify_node` still sees the previous itinerary, skips its "no plan produced" guard and calls `PlannerOutput.model_validate(None)`, which raises. The planned fix is to guard on `planner_output` itself so the request degrades to the fallback plan. *Check whether this has been fixed and re-run before the evaluation; if so, update the 10/12 figure.*

7. Known gaps (say these before they are asked)

- Playwright authenticated journeys are skipped (no scripted Keycloak login yet).
 - Performance and load baselines cover one page or route on a local dev server, with no SLA yet.
 - The AI backend's network isolation is checked manually only.
 - Coverage is not gated in CI.
 - Golden scenarios are run manually against the live stack before release, not in CI. The LLM free-tier limits make that unreliable, and Groq's token ceiling is a real capacity limit.
 - No staging environment. The local Docker Compose stack stands in for it.
-

8. Suggested talking points (about 2 minutes)

1. "We test **per service and per boundary**, using three independent test runners with one shared philosophy."
 2. "**637 automated tests, all green in CI**: Vitest and Playwright on the JS side, pytest on the AI side."
 3. "Because the AI is non-deterministic, we test **properties** (valid schema, within budget, correct fallback source), not exact text."
 4. "Security focus is **data isolation**, proven by 404-on-foreign-owner tests and a real JWT guard chain e2e."
 5. "A **12-scenario live golden test scored 10/12**. It surfaced a real orchestrator bug, and we report the measured number."
 6. "Testing found real bugs: a silent Prisma filter overwrite, a broken harness, and CI misconfiguration."
 7. "We are upfront about gaps: authenticated UI journeys, load and performance baselines, and the AI backend's network isolation."
-

9. Where the tests live

Paths are relative to `D:\Smart_Tourism`.

Backend (NestJS, Vitest)

Kind	Location	Run with
Unit tests (*.spec.ts , beside the code)	backend/src/admin/ (5 specs, plus 1 in admin/dto/), backend/src/auth/ (4), backend/src/chat/ (2), backend/src/budget/ , explore/ , health/ , trips/ , users/ (1 each), plus 1 directly in backend/src/	npm test
End-to-end tests	backend/test/access-control.e2e-spec.ts , security.e2e- spec.ts , app.e2e-spec.ts	npm run test:e2e
E2E config	backend/vitest.config.e2e.ts	
CI	backend/.github/workflows/ci.yml	

Frontend (Next.js)

Kind	Location	Run with
Unit tests (Vitest + React Testing Library)	frontend-web/src/lib/__tests__/ (auth.spec.ts , authz.spec.ts , trip- mappers.spec.ts , trip-store.spec.ts) and frontend- web/src/components/budget/__tests__/SpendByCategoryPanel.spec.tsx	npm test
Playwright journeys	frontend-web/e2e/journey.spec.ts (config: frontend- web/playwright.config.ts)	npm run test:e2e
CI	frontend-web/.github/workflows/ci.yml	

AI backend (FastAPI, pytest)

Kind	Location	Run with
Unit tests	ai-backend/tests/	python -m pytest -m "not external" -q
Golden scenarios (12, live stack)	ai-backend/scripts/e2e_check.py	python scripts/e2e_check.py
LLM failover reliability	ai- backend/scripts/check_llm_chain_reliability.py	
CI	ai-backend/.github/workflows/ci.yml	

Test documentation

Documentation/Testing/: SmartJourney_Master_Test_Plan.docx , MASTER_TEST_PLAN.md ,
OWASP_SECURITY_MAPPING.md , CORRECTIONS.md , and this brief.

Numbers to re-check before presenting

- `ai-backend/tests/` now contains 60 test files, but the plan (2026-09-27) says 42 files and 544 tests. Re-run the suite and use the fresh count.
- `backend/test/` holds 3 e2e spec files, while the plan reports 6 e2e tests. Those are probably 6 test cases across the 3 files, but confirm with `npm run test:e2e`.